

Hands-On Lab: Kubernetes RBAC

Role-Based Access Control — Roles, ClusterRoles & Bindings in Practice

In this lab you will configure fine-grained, least-privilege access to a Kubernetes cluster using Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings. You will create service accounts, grant scoped permissions, verify access with `kubectl auth can-i`, and apply real-world access patterns used in production clusters.

Duration	45–60 minutes
Level	Intermediate
Prerequisites	Running cluster (minikube / kind / cloud), kubectl configured with cluster-admin access
Tools needed	kubectl v1.24+

Learning Objectives

- Explain the difference between Roles/RoleBindings (namespaced) and ClusterRoles/ClusterRoleBindings (cluster-wide)
- Write RBAC manifests using apiGroups, resources, verbs, and subjects
- Grant a ServiceAccount read-only access to Pods in a single namespace
- Grant a group cluster-wide read access using an aggregated ClusterRole
- Verify effective permissions with `kubectl auth can-i` and impersonation
- Recognize common RBAC misconfigurations and apply least-privilege best practices

Part 1 — Core Concepts

Kubernetes RBAC uses four API objects to control who (subject) can do what (verb) on which resource, and where (namespace scope).

Object	Scope	Purpose
Role	Namespace	Defines permissions (verbs on resources) within one namespace
ClusterRole	Cluster-wide	Defines permissions across all namespaces, or on cluster-scoped resources (nodes, PVs) and non-resource URLs
RoleBinding	Namespace	Grants a Role (or ClusterRole) to subjects, scoped to one namespace
ClusterRoleBinding	Cluster-wide	Grants a ClusterRole to subjects across the entire cluster

A useful mental model: a **Role/ClusterRole is a stencil of permissions**; a **Binding is what glues that stencil to an identity**. A Role by itself grants nothing until it is bound to a subject.

Subjects

- **User** — a human identity, authenticated externally (certs, OIDC token, etc.); Kubernetes has no User object
- **Group** — a set of users, also managed externally
- **ServiceAccount** — an in-cluster identity for Pods and controllers, the only subject type Kubernetes manages natively

Verbs & Resources

A rule combines **apiGroups**, **resources**, and **verbs**. Common verbs:

get	Read a single object	list	Read a collection
watch	Stream changes	create	Create new objects
update	Replace an object	patch	Partially modify
delete	Remove an object	deletecollection	Remove a collection

Example Role manifest:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: payments
  name: pod-reader
rules:
- apiGroups: [""]          # "" = core API group
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

The apiGroups: [""] entry refers to the core API group (Pods, Services, ConfigMaps, Secrets, etc.). Named groups like apps or batch are required for resources such as Deployments or Jobs.

Part 2 — Hands-On Exercises

Work through the tasks below in order. Each task builds the next. Run all commands against a test cluster — never a production cluster.

Setup

```
kubectl create namespace payments
kubectl create serviceaccount pod-viewer -n payments
```

TASK 1 Grant namespace-scoped read-only access

Create a Role that allows read-only access to Pods and Pod logs in the payment's namespace, then bind it to the pod-viewer ServiceAccount.

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: payments
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["pods/log"]
  verbs: ["get"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: payments
subjects:
- kind: ServiceAccount
  name: pod-viewer
  namespace: payments
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
EOF
```

✓ Expected result

`kubectl auth can-i list pods -n payments --as=system:serviceaccount:payments:pod-viewer → yes`

`kubectl auth can-i delete pods -n payments --as=system:serviceaccount:payments:pod-viewer → no`

`kubectl auth can-i list pods -n default --as=system:serviceaccount:payments:pod-viewer → no` (Role is namespace-scoped)

TASK 2 Grant cluster-wide access with a ClusterRole

Some resources (Nodes, PersistentVolumes, Namespaces) are cluster-scoped and cannot be targeted by a namespaced Role. Create a ClusterRole for read-only Node access and bind it to a group called sre-team.

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: node-reader-binding
subjects:
- kind: Group
  name: sre-team
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
EOF
```

✓ Expected result

kubectl auth can-i list nodes --as=alice --as-group=sre-team → **yes**

kubectl auth can-i list nodes --as=alice → **no** (group membership required)

TASK 3 Reuse a ClusterRole inside one namespace

A ClusterRole can also be bound with a **RoleBinding** to grant its permissions in just one namespace — useful for reusing a common permission set (e.g. the built-in view ClusterRole) without duplicating rules.

```
kubectl create rolebinding dev-view \
  --clusterrole=view \
  --serviceaccount=payments:pod-viewer \
  --namespace=payments
```

✓ Expected result

kubectl auth can-i list configmaps -n payments --as=system:serviceaccount:payments:pod-viewer → **yes**

```
kubectl auth can-i list configmaps -n kube-system --as=system:serviceaccount:payments:pod-viewer → no
```

TASK 4 Least-privilege access for a CI/CD pipeline

Real-world pattern: a CI/CD ServiceAccount needs to roll out Deployments but should never read Secrets or delete namespaces. Create a scoped Role covering only what it needs.

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: payments
  name: ci-deployer
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "update", "patch"]
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ci-deployer-binding
  namespace: payments
subjects:
- kind: ServiceAccount
  name: ci-bot
  namespace: payments
roleRef:
  kind: Role
  name: ci-deployer
  apiGroup: rbac.authorization.k8s.io
EOF
kubectl create serviceaccount ci-bot -n payments --dry-run=client -o yaml | kubectl apply -f -
```

✓ Expected result

`kubectl auth can-i update deployments -n payments --as=system:serviceaccount:payments:ci-bot → yes`

`kubectl auth can-i get secrets -n payments --as=system:serviceaccount:payments:ci-bot → no`

`kubectl auth can-i delete namespaces --as=system:serviceaccount:payments:ci-bot → no`

TASK 5 Audit effective permissions

Before granting broader access, always check what a subject can already do. Use these commands to audit and self-check permissions.

```
# Check a specific permission
kubectl auth can-i create deployments -n payments \
  --as=system:serviceaccount:payments:ci-bot

# List everything the current user can do in a namespace
kubectl auth can-i --list -n payments

# List everything a specific ServiceAccount can do
kubectl auth can-i --list \
  --as=system:serviceaccount:payments:ci-bot -n payments

# Inspect all RoleBindings in a namespace
kubectl get rolebindings,roles -n payments -o wide

# Find every ClusterRoleBinding for a given subject
kubectl get clusterrolebindings -o json | \
  jq '.items[] | select(.subjects[]?.name=="sre-team")'
```

Cleanup

```
kubectl delete namespace payments
kubectl delete clusterrole node-reader
kubectl delete clusterrolebinding node-reader-binding
```


Part 3 — Real-World Access Patterns

Persona	Pattern
Developer (read-only)	Bind the built-in view ClusterRole via a RoleBinding per namespace they own
Namespace admin	Bind the built-in admin ClusterRole via a RoleBinding — full control within the namespace, no cluster-scoped access
CI/CD pipeline	Custom Role limited to Deployments/Pods/ConfigMaps in one namespace; never grant Secrets or RBAC-editing verbs
Monitoring agent	ClusterRole with get/list/watch on pods, nodes, and metrics endpoints; read-only, cluster-wide
Platform / SRE team	ClusterRole aggregation combining several ClusterRoles via aggregationRule labels

Best Practices

- **Principle of least privilege** — grant only the verbs and resources a subject actually needs
- **Prefer Roles over ClusterRoles** when access can be scoped to a namespace
- **One ServiceAccount per workload**, not one shared account for everything
- **Never bind to the default ServiceAccount** — it exists in every namespace and is easy to over-grant by accident
- **Avoid wildcards** (resources: [""], verbs: [""]) outside break-glass admin roles
- **Treat RBAC-editing verbs as sensitive** — the ability to create Roles/Bindings/escalate is itself a privilege
- **Audit regularly** with `kubectl auth can-i --list` and tools like `rbac-lookup` or `kubectl-who-can`
- **Store RBAC manifests in version control** and review changes like code

Common Pitfalls

Pitfall	Why it's a problem
Using a ClusterRoleBinding when a RoleBinding would do	Grants access in every namespace, including future ones
Granting <code>get secrets</code> broadly	Secrets often contain credentials for other systems — treat as high-sensitivity
Granting <code>create on clusterrolebindings / roles</code>	Allows privilege escalation — the subject can grant itself more access

Forgetting pods/exec and pods/log are separate resources from pods	A "read-only" role can still allow shelling into containers if pods/exec isn't excluded
--	---

Knowledge Check

Answer briefly, then compare against the answer key on the next page.

- 1. What is the difference in scope between a Role and a ClusterRole?
- 2. Which object actually grants access to a subject: the Role or the RoleBinding?
- 3. Can a RoleBinding reference a ClusterRole? What does that achieve?
- 4. Why is granting create on RBAC objects dangerous?
- 5. Which subject type does Kubernetes manage natively — User, Group, or ServiceAccount?

Answer Key

- **1.** A Role's rules apply only within its own namespace; a ClusterRole's rules apply cluster-wide (or to cluster-scoped resources such as Nodes) unless bound with a RoleBinding, which limits it to one namespace.
- **2.** The RoleBinding (or ClusterRoleBinding). A Role/ClusterRole only defines permissions; it grants nothing until bound to a subject.
- **3.** Yes — a RoleBinding can reference a ClusterRole via `roleRef`. This lets you reuse a common permission set (e.g. the built-in `view` ClusterRole) while limiting its effect to a single namespace.
- **4.** A subject that can create or modify Roles/Bindings can grant itself additional permissions, effectively escalating its own privileges beyond what was originally intended.
- **5.** ServiceAccount — Users and Groups are authenticated by an external identity provider and are not first-class API objects in Kubernetes.